

Atividade 1 - Python e Java

Python

- a) Saída: [1] e [1, 2]
- b) Explicação: Isso acontece porque a lista foi criada como valor padrão do parâmetro, e ela só é criada uma vez. Aí quando eu chamo a função de novo ela usa a mesma lista de antes e só adiciona o 2 nela.

Java

- a) Saída: 0 5
- b) Explicação: O array foi mudado dentro da função, então o primeiro valor dele virou 0 lá fora também. Já o n continua 5, porque ele é passado como cópia e mexer nele dentro da função não muda o original.

Atividade 2 - Python e C

Python

- a) Saída: [2, 2, 2]
- b) Explicação: As funções lambda guardam a variável i e não o valor dela. Quando o laço termina o i fica valendo 2, então quando eu chamo as funções todas devolvem 2.

C

- a) Saída: 3
- b) Explicação: O n foi declarado como static, então ele não perde o valor quando a função termina. Como a função foi chamada 3 vezes, o contador foi somando até chegar em 3.

Atividade 3 - Rust e Python

Rust

- a) Saída: [1, 2, 3] [2, 4, 6]
- b) Explicação: A função pega o vetor original e cria um novo com cada valor multiplicado por 2. Como usou iter(), o vetor original só é lido e não é alterado, por isso ele continua [1, 2, 3].

Python

- a) Saída: UnboundLocalError
- b) Explicação: O erro acontece porque a função tenta usar o total antes de ter dado um valor pra ele dentro dela. Como tem uma atribuição total na função, o Python trata ele como variável local, mesmo existindo outro total fora.